

Packet Sniffing Attack — Final Report

CSE406: Cyber Security Sessional · Group 10 · Attack Tool 2: Packet sniffing attack and sniff HTTP/Telnet passwords
2105128 – Nakib Arman • 2105143 – Fatin Ishrak Arian

a. Steps of the Attack

Topology. Three VirtualBox VMs (Ubuntu Server 24.04.5), one shared VirtualBox **Internal Network** (`sniffnet` , 192.168.56.0/24):

Host	Role	IP
victim	HTTP + Telnet client	192.168.56.11
server	HTTP Basic-Auth (:80) + Telnet daemon (:23), account <code>bob:hunter2</code>	192.168.56.12
attacker	Raw-socket sniffer, NIC in Promiscuous Mode “Allow All”	192.168.56.13

Promiscuous Mode on the attacker's adapter is what makes VirtualBox's virtual switch (which otherwise learns MAC addresses like a real switch) deliver a copy of the victim ↔ server conversation to the attacker — reproducing the hub / mirrored-switch-port assumption from the Design Report.

Procedure

- `server` runs `server/http_server.py` and the system `telnetd` .
- `attacker` runs `sniffer/sniffer.py enp0s8` as root: opens a raw `AF_PACKET` socket and manually unpacks Ethernet / IPv4 / TCP headers with Python's `struct` module (no scapy / libpcap). Purely passive — it only calls `recvfrom()` , never `send()` .
- `victim` performs a normal HTTP Basic-Auth login, then a normal interactive Telnet login.

```
Escape character is '^['.
Linux 6.8.0-139-generic (server) (pts/0)
server login: bob
Password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Sep 14 01:21:30 PM UTC 2026

System load:          0.16
Usage of /:           26.2% of 11.43GB
Memory usage:        12%
Swap usage:          0%
Processes:           163
Users logged in:      1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe5b:1a83

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

bob@server:~$ _
```

Victim's own terminal: a completely ordinary `telnet` session — banner, `login:`, `Password:`, shell prompt. Nothing on this screen indicates it is being observed.

```
13:11:13.309 TELNET 08:00:27:90:32:77/192.168.56.11:37464 -> 08:00:27:d9:77:a0/192.168.56.12:23
13:11:13.309 TELNET 08:00:27:d9:77:a0/192.168.56.12:23 -> 08:00:27:90:32:77/192.168.56.11:37464
13:11:13.309 TELNET 08:00:27:90:32:77/192.168.56.11:37464 -> 08:00:27:d9:77:a0/192.168.56.12:23
13:11:13.558 TELNET 08:00:27:d9:77:a0/192.168.56.12:23 -> 08:00:27:90:32:77/192.168.56.11:37464
13:11:13.558 TELNET 08:00:27:90:32:77/192.168.56.11:37464 -> 08:00:27:d9:77:a0/192.168.56.12:23
13:11:13.558 TELNET 08:00:27:90:32:77/192.168.56.11:37464 -> 08:00:27:d9:77:a0/192.168.56.12:23
13:11:13.559 TELNET 08:00:27:d9:77:a0/192.168.56.12:23 -> 08:00:27:90:32:77/192.168.56.11:37464
13:11:13.559 TELNET 08:00:27:90:32:77/192.168.56.11:37464 -> 08:00:27:d9:77:a0/192.168.56.12:23
ubuntu@attacker:~$ _
```

Attacker's console, live-tailing `sniffer_session.log` during the same Telnet session — every frame between victim and server is captured in real time.

Recovered by the sniffer, from the same run:

```
*** HTTP credentials recovered: bob:hunter2 ***
```

```
*** Telnet credentials recovered: bob:hunter2 ***
```

How the attacker actually recovers the username and password

The sniffer never sees a “username” or “password” field as such — it only ever sees raw bytes on the wire. Recovery happens in two different ways depending on protocol, both implemented in `sniffer/sniffer.py` :

- **HTTP** — the victim's client sends the header `Authorization: Basic <base64>` on every request once authenticated. The sniffer's `handle_http()` buffers each TCP stream's bytes, regex-searches for that header, and simply **base64-decodes** the value — `bob:hunter2` is right there in the decoded text, because Basic-Auth is *encoded*, not encrypted.
- **Telnet** — there is no single “credentials packet” at all. The server sends a `login:` prompt, the client's keystrokes for the username arrive as individual TCP segments (often one byte each), then the server sends `Password:` , then the password keystrokes arrive the same way. The sniffer's `handle_telnet()` watches the server → client direction for those two prompts

to know which stage it's in, buffers the client → server bytes until a line terminator, and hands the first completed line to `telnet_username` and the second to the final recovered pair.

```
18:17:49.246 HTTP 08:00:27:d9:77:a0/192.168.56.12:80 -> 08:00:27:90:32:77/192.168.56.11:40328
18:17:49.246 HTTP 08:00:27:90:32:77/192.168.56.11:40328 -> 08:00:27:d9:77:a0/192.168.56.12:80
18:17:49.246 HTTP 08:00:27:90:32:77/192.168.56.11:40328 -> 08:00:27:d9:77:a0/192.168.56.12:80
[18:17:49.246 HTTP 08:00:27:90:32:77/192.168.56.11:40328 -> 08:00:27:d9:77:a0/192.168.56.12:80] ***
HTTP credentials recovered: bob:hunter2 ***
--
18:18:22.033 TELNET 08:00:27:90:32:77/192.168.56.11:43874 -> 08:00:27:d9:77:a0/192.168.56.12:23
18:18:22.034 TELNET 08:00:27:d9:77:a0/192.168.56.12:23 -> 08:00:27:90:32:77/192.168.56.11:43874
18:18:22.084 TELNET 08:00:27:90:32:77/192.168.56.11:43874 -> 08:00:27:d9:77:a0/192.168.56.12:23
[18:18:22.084 TELNET 08:00:27:90:32:77/192.168.56.11:43874 -> 08:00:27:d9:77:a0/192.168.56.12:23] ***
* Telnet credentials recovered: bob:hunter2 ***
ubuntu@attacker:~$ _
```

The sniffer's own log, filtered to show exactly this happening: raw per-frame capture lines immediately followed by the extraction result, for both protocols in the same run. The bracketed `[...]` line is not a separate event — it is the same frame as the raw line above it, re-printed with the decoded/reassembled credential once the parser recognizes it completes a login. Nothing is guessed or brute-forced; every byte was already sitting in the payload the attacker's promiscuous NIC received.

Is the password visible in the raw `tcpdump` capture itself, without the sniffer? Depends on the protocol — checked directly against `docs/captures/capture.pcap` :

```
$ strings capture.pcap | grep -i authorization
Authorization: Basic Ym9iOmh1bnRlcjI=
$ echo Ym9iOmh1bnRlcjI= | base64 -d
bob:hunter2
```

For **HTTP**, yes — the whole header, `bob:hunter2` included, sits as one contiguous string in the raw capture. Base64 isn't encryption; any tool that can `strings / grep` a pcap and pipe the match through `base64 -d` recovers it, no custom sniffer required.

```
$ strings capture.pcap | grep -c hunter2
0
```

For **Telnet**, the string `hunter2` never appears as one contiguous run of bytes anywhere in the capture — real Telnet sends one keystroke per TCP packet, so the password is scattered as seven separate single-byte packets (`h`, `u`, `n`, `t`, `e`, `r`, `2`). It is fully present in the capture (nothing is missing — see the raw per-frame lines in the screenshot above, and Wireshark's **Follow** → **TCP Stream** will visually reassemble it for a human), but no simple string search finds it. This is exactly why `sniffer.py`'s `handle_telnet()` exists: it buffers the client → server bytes packet-by-packet and reassembles them into a line itself, rather than relying on the credential already being one searchable string the way HTTP's is.

b. Was the Attack Successful? Why / Why Not?

Yes, fully successful against both target protocols, judged against the two criteria set out in the Design Report:

- **Correctness** — the recovered `bob:hunter2` exactly matches what the victim typed, for both HTTP and Telnet, reproduced across multiple independent runs.
- **Non-interference** — the victim's HTTP request (`200 OK`) and Telnet session (full shell access) completed identically with or without the sniffer running; the attacker transmits nothing toward either host.

Why it worked: HTTP Basic-Auth and Telnet both transmit credentials as clear text inside the TCP payload — HTTP as a base64-encoded (not encrypted) `Authorization: Basic` header, Telnet as raw keystrokes split across one-byte TCP segments. Once a NIC receives a copy of those frames, recovering the credentials needs no cryptography to be broken: just a base64 decode for HTTP, and reassembling the per-keystroke stream between the `login: / Password:` prompts for Telnet.

Cross-checked against an independent `tcpdump` capture taken concurrently on the attacker (`docs/captures/capture.pcap`, 110 packets) — identical source/destination IPs, ports and timestamps to the sniffer's own log, confirming the sniffer is reading genuine wire traffic rather than fabricating results.

c. Observed Output

Host	Observed output
Attacker	One log line per captured HTTP/Telnet frame (timestamp, src/dst MAC + IP + port, protocol), plus a highlighted <code>*** ... credentials recovered ***</code> line per successful login. Full log: <code>docs/captures/sniffer_session.log</code> .
Victim	HTTP: Server responded <code>200: Welcome, authenticated user.</code> Telnet: ordinary banner → <code>login:</code> → <code>Password:</code> → shell prompt, with no indication of observation.
Server	Served both logins normally; <code>ss -tln</code> shows only that it is listening on <code>:80 / :23</code> (and, after the defense was added, <code>:22</code>) — no awareness of the attacker.

```
LISTEN 0      5      0.0.0.0:80    0.0.0.0:*
LISTEN 0      4096    0.0.0.0:22    0.0.0.0:*
LISTEN 0      4096    [::]:22       [::]:*
LISTEN 0      64      *:23          *:*
```

ubuntu@server:~\$ _

Server VM: HTTP (:80), SSH (:22, added for the defense demo) and Telnet (:23) all listening normally.

d. Countermeasure (Bonus)

Per the Design Report's own proposal: the direct, complete defense is protocol-level encryption, not attempting to detect the sniffer — a purely passive listener generates no network traffic of its own and is inherently undetectable. We implemented and tested this by replacing Telnet with **SSH** for the same login.

Implementation

- Installed `openssh-server` on `server` (listens on `:22` automatically).
- Same account, `ssh bob@192.168.56.12` in place of `telnet 192.168.56.12` — no other change to the topology.
- Left `sniffer.py` and a parallel `unfiltered tcpdump` running throughout, to test whether the attack still recovers anything.

```
bob@server:~$ exit
logout
Connection to 192.168.56.12 closed.
bob@server:~$ ssh bob@192.168.56.12
bob@192.168.56.12's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Sep 14 05:22:26 PM UTC 2026

System load:            0.0
Usage of /:              26.2% of 11.43GB
Memory usage:           12%
Swap usage:             0%
Processes:              167
Users logged in:        2
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe5b:1a83

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Mon Sep 14 17:22:27 2026 from 192.168.56.12
bob@server:~$ _
```

Victim logging in over SSH — functionally identical experience to the Telnet login, but the wire content is now encrypted.

Result

`sniffer.py` recovered nothing — it only watches ports 80/23. More rigorously, we searched the *raw captured bytes* of the full SSH session (`docs/captures/ssh_defense_capture.pcap` , 44 packets) for the literal credential text:

```
$ strings ssh_defense_capture.pcap | grep -c hunter2
0
$ strings ssh_defense_capture.pcap | grep -c '^bob$'
0
```

versus trivial recovery in the unencrypted capture. The captured bytes are genuine SSH-protocol ciphertext — the attacker still sees every frame, but none of it is usable.

Conclusion

Encrypting the transport (SSH in place of Telnet; the same argument applies to HTTPS in place of HTTP) is a complete, protocol-level defense against this entire class of passive sniffing attack. It costs nothing in usability — the legitimate login still works identically — while making the captured payload permanently unreadable to any observer on the segment.

Artifacts

- `docs/SETUP.md` — VM creation & networking (VBoxManage unattended install)
- `docs/RUNNING.md` — local (non-VM) smoke-test instructions
- `docs/experiment-results.md` — condensed technical run summary
- `docs/defense.md` — this countermeasure, in full detail
- `docs/captures/` — `capture.pcap` (HTTP+Telnet, Wireshark-ready), `ssh_defense_capture.pcap` , `sniffer_session.log`
- `docs/screenshots/` — all screenshots referenced above, full resolution
- `scripts/` — one-command wrappers for both the local test and the full 3-VM demo
- `sniffer/sniffer.py` , `server/` , `victim/` — full source of the attack tool and demo harness